

# Relazione Laboratorio HPC – Esercitazione 2

## Timing, Matrix Multiplication, Network e Storage

Federico Becchi

Giugno 2026

## Indice

|          |                                                       |          |
|----------|-------------------------------------------------------|----------|
| <b>1</b> | <b>Ambienti di calcolo</b>                            | <b>3</b> |
| <b>2</b> | <b>Timing</b>                                         | <b>3</b> |
| 2.1      | Descrizione del programma . . . . .                   | 3        |
| 2.2      | Script di esecuzione . . . . .                        | 3        |
| 2.2.1    | PC – <code>timing.sh</code> . . . . .                 | 3        |
| 2.2.2    | Cluster – <code>timing.slurm</code> . . . . .         | 3        |
| 2.3      | Risultati . . . . .                                   | 4        |
| 2.4      | Considerazioni . . . . .                              | 4        |
| <b>3</b> | <b>Matrix Multiplication</b>                          | <b>4</b> |
| 3.1      | Descrizione . . . . .                                 | 4        |
| 3.2      | CPU – Versione seriale e GSL . . . . .                | 5        |
| 3.2.1    | PC – <code>mm.sh</code> . . . . .                     | 5        |
| 3.2.2    | Cluster – <code>omp_mm_scaling.slurm</code> . . . . . | 5        |
| 3.3      | Risultati CPU . . . . .                               | 5        |
| 3.4      | OpenMP – scaling . . . . .                            | 6        |
| 3.4.1    | Script <code>omp_mm_scaling.slurm</code> . . . . .    | 6        |
| 3.4.2    | Risultati OpenMP . . . . .                            | 6        |
| 3.5      | Grafici CPU . . . . .                                 | 6        |
| 3.6      | Considerazioni CPU . . . . .                          | 7        |
| 3.7      | GPU – CUDA . . . . .                                  | 7        |
| 3.7.1    | Descrizione . . . . .                                 | 7        |
| 3.7.2    | Script <code>matrixMul.slurm</code> . . . . .         | 7        |
| 3.7.3    | Risultati GPU . . . . .                               | 7        |
| 3.8      | Confronto CPU vs GPU . . . . .                        | 8        |
| <b>4</b> | <b>Bandwidth e Latency di Rete</b>                    | <b>8</b> |
| 4.1      | Descrizione . . . . .                                 | 8        |
| 4.2      | Script di esecuzione . . . . .                        | 9        |
| 4.2.1    | PC – Docker <code>band_lat.sh</code> . . . . .        | 9        |
| 4.2.2    | Cluster – <code>band_lat.slurm</code> . . . . .       | 9        |
| 4.3      | Risultati – Bandwidth . . . . .                       | 10       |
| 4.4      | Risultati – Latency . . . . .                         | 10       |
| 4.5      | Considerazioni . . . . .                              | 11       |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>5</b> | <b>Disk Test</b>                                | <b>11</b> |
| 5.1      | Descrizione . . . . .                           | 11        |
| 5.2      | PC – <code>disktest.sh</code> . . . . .         | 12        |
| 5.3      | Cluster – <code>disktest.slurm</code> . . . . . | 12        |
| 5.4      | Risultati . . . . .                             | 12        |
| 5.5      | Considerazioni . . . . .                        | 13        |
| <b>6</b> | <b>Conclusioni</b>                              | <b>13</b> |

# 1 Ambienti di calcolo

I test sono stati eseguiti su due ambienti distinti:

- **PC locale:** MacBook Air con chip Apple M3, macOS 26.3.1, architettura ARM64
- **Cluster HPC Ateneo:** nodo `ui01.hpc.unipr.it`, Intel Xeon Gold 6238R @ 2.20 GHz, architettura x86\_64 con supporto AVX512, gestito tramite SLURM

## 2 Timing

### 2.1 Descrizione del programma

Il programma `timing.c` misura due tipologie di tempo distinte tramite la funzione `clock_gettime`:

- **REALTIME** (`CLOCK_REALTIME`): il tempo di parete (*wall-clock time*), ossia il tempo reale trascorso dall'inizio alla fine del programma.
- **CPUTIME** (`CLOCK_PROCESS_CPUTIME_ID`): il tempo effettivo di utilizzo della CPU da parte del processo.

Il programma esegue due operazioni sequenziali: una `sleep(1)` (che blocca il processo senza consumare CPU) e un ciclo di 500 milioni di iterazioni floating point. Si aspetta quindi che:

$$\text{REALTIME} \approx 1\text{ s (sleep)} + t_{\text{loop}} > \text{CPUTIME} \approx t_{\text{loop}}$$

### 2.2 Script di esecuzione

#### 2.2.1 PC – `timing.sh`

```
1 #!/bin/sh
2 gcc timing.c -o timing -O2
3 ./timing > timing_pc.csv
```

#### 2.2.2 Cluster – `timing.slurm`

```
1 #!/bin/sh
2 #SBATCH --partition=cpu_guest
3 #SBATCH --qos=cpu_guest
4 #SBATCH --nodes=1
5 #SBATCH --cpus-per-task=1
6 #SBATCH --time=0-00:05:00
7
8 gcc -O2 timing.c -o timing
9 ./timing > timing.csv
```

## 2.3 Risultati

| Ambiente                  | REALTIME (s) | CPUTIME (s) |
|---------------------------|--------------|-------------|
| PC (MacBook Air M3)       | 1.3803       | 0.3752      |
| Cluster (Xeon Gold 6238R) | 1.9682       | 0.9680      |

Tabella 1: Risultati del test di timing su PC e Cluster.

## 2.4 Considerazioni

In entrambi gli ambienti si osserva che  $\text{REALTIME} > \text{CPUTIME}$ , confermando il comportamento atteso: il secondo trascorso in `sleep(1)` incrementa solo il wall-clock time, non il tempo CPU.

Il CPUTIME sul PC (0.3752 s) è inferiore rispetto al cluster (0.9680 s): il chip Apple M3 con architettura ARM64 e frequenza boost più alta esegue il ciclo floating point in modo più efficiente. Il REALTIME del cluster è più alto perché include, oltre all'`sleep`, un loop più lento.

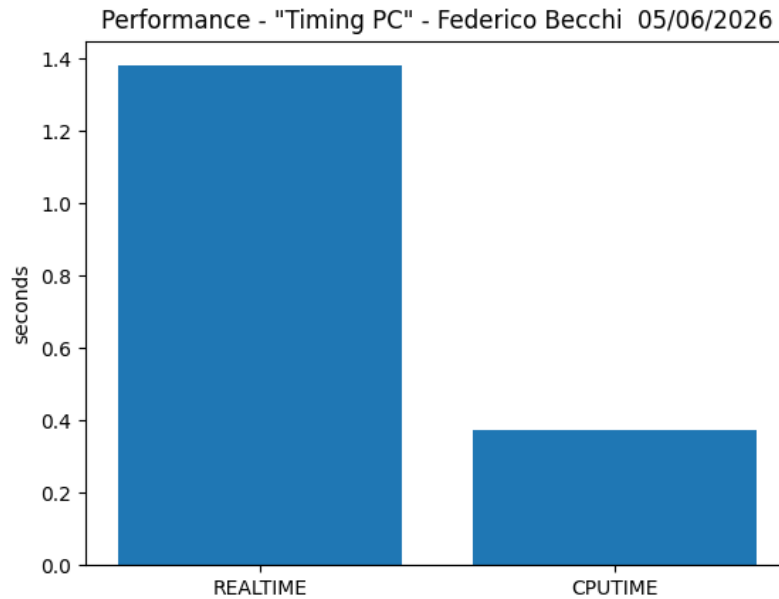


Figura 1: Grafico comparativo REALTIME vs CPUTIME su PC e Cluster.

# 3 Matrix Multiplication

## 3.1 Descrizione

L'esercizio esplora le prestazioni della moltiplicazione matriciale  $C = A \times B$  con matrici quadrate di dimensione  $N = 1024$  in singola precisione floating point (`float`). Vengono confrontate diverse ottimizzazioni:

- Implementazione seriale con **GCC/ICC** (cicli annidati)

- Ottimizzazione tramite libreria **GSL** (`gsl_blas_sgemm`)
- Parallelizzazione **OpenMP**
- Accelerazione **GPU** (CUDA)

La metrica di prestazione usata è **GFLOP/s**, calcolata come:

$$\text{GFLOP/s} = \frac{N^3 \times 2}{10^9 \times \text{tempo(s)}}$$

Per  $N = 1024$ : GFLOP  $\approx 2.15$ .

## 3.2 CPU – Versione seriale e GSL

### 3.2.1 PC – `mm.sh`

Per compilare su macOS è stato necessario installare la libreria GSL tramite Homebrew:

```
1 brew install gsl
```

e adattare i path nella compilazione:

```
1 gcc $OPT -o mm mm.c \
2   -I/opt/homebrew/opt/gsl/include \
3   -L/opt/homebrew/opt/gsl/lib \
4   -lgsl -lgslcblas
```

Lo script esegue il programma due volte: una senza GSL (cicli naive) e una con l'opzione `-g` (routine BLAS ottimizzata).

### 3.2.2 Cluster – `omp_mm_scaling.slurm`

Sul cluster è disponibile anche il compilatore Intel (`icc`) che sfrutta le istruzioni AVX/AVX512 del processore Xeon. I test includono combinazioni di compilatori (`gcc/icc`), ottimizzazioni vettoriali (AVX) e GSL.

## 3.3 Risultati CPU

| Ambiente | Configurazione  | Tempo (s) | GFLOP/s |
|----------|-----------------|-----------|---------|
| PC       | gcc (seriale)   | 0.65      | 3.31    |
| PC       | gcc + GSL       | 0.30      | 7.09    |
| Cluster  | gcc (seriale)   | 1.42      | 1.52    |
| Cluster  | gcc + GSL       | 0.10      | 22.13   |
| Cluster  | icc (seriale)   | 1.42      | 1.52    |
| Cluster  | icc + GSL       | 0.10      | 22.10   |
| Cluster  | icc + AVX       | 0.18      | 12.20   |
| Cluster  | icc + AVX + GSL | 0.10      | 22.19   |

Tabella 2: Prestazioni moltiplicazione matriciale CPU,  $N = 1024$ , singola precisione.

### 3.4 OpenMP – scaling

#### 3.4.1 Script omp\_mm\_scaling.slurm

```

1 #SBATCH --cpus-per-task=32
2
3 for N in 1024; do
4   for NT in 1 2 4 8; do
5     ./omp_mm -n $N -t $NT -c $HOSTNAME-$NT
6   done
7 done > $FILENAME

```

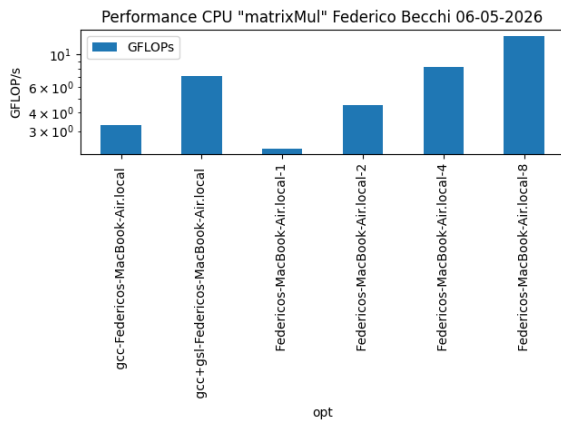
Il programma `omp_mm.c` parallelizza i cicli di inizializzazione e calcolo tramite la direttiva `#pragma omp parallel for`.

#### 3.4.2 Risultati OpenMP

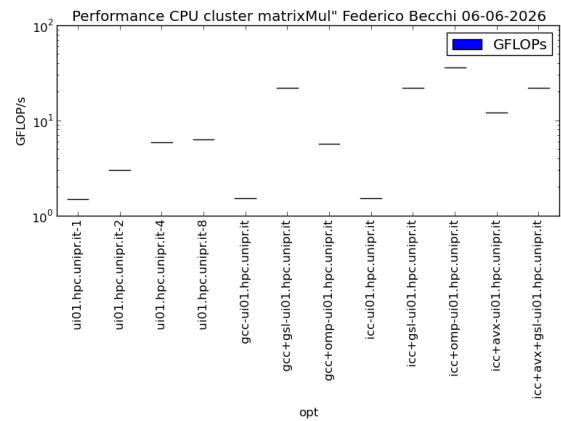
| Ambiente | Thread | Tempo (s) | GFLOP/s | Speedup |
|----------|--------|-----------|---------|---------|
| PC       | 1      | 0.94      | 2.27    | 1.00    |
| PC       | 2      | 0.47      | 4.53    | 2.00    |
| PC       | 4      | 0.26      | 8.29    | 3.62    |
| PC       | 8      | 0.16      | 13.49   | 5.88    |
| Cluster  | 1      | 1.43      | 1.50    | 1.00    |
| Cluster  | 2      | 0.72      | 2.99    | 1.99    |
| Cluster  | 4      | 0.36      | 5.89    | 3.97    |
| Cluster  | 8      | 0.34      | 6.35    | 4.21    |

Tabella 3: Scaling OpenMP per la moltiplicazione matriciale,  $N = 1024$ .

### 3.5 Grafici CPU



(a) PC (MacBook Air M3)



(b) Cluster (Xeon Gold 6238R)

Figura 2: Prestazioni CPU (GFLOP/s) con diverse ottimizzazioni,  $N = 1024$ .

### 3.6 Considerazioni CPU

- La versione seriale naive è più lenta sul cluster rispetto al PC, probabilmente per via della frequenza di clock più alta del chip M3 e della migliore cache locality.
- L'uso di GSL porta un enorme vantaggio su entrambi gli ambienti: sul cluster si passa da 1.52 a 22.13 GFLOP/s (+1357%), perché GSL invoca routine BLAS altamente ottimizzate con vettorizzazione AVX.
- Lo speedup OpenMP sul PC è quasi lineare fino a 4 thread; a 8 thread si riduce (5.88x invece di 8x ideale) per contention sulla memoria condivisa.
- Sul cluster a 8 thread lo speedup si ferma a 4.21x: il nodo assegnato nella partizione `cpu_guest` ha limitazioni sulle risorse fisiche disponibili.

### 3.7 GPU – CUDA

#### 3.7.1 Descrizione

Il programma `matrixMul` è il sample CUDA di NVIDIA che esegue la moltiplicazione matriciale su GPU tramite kernel CUDA ottimizzato con tiling (*shared memory*). Le matrici  $A$  ( $1024 \times 1024$ ) e  $B$  ( $1024 \times 1024$ ) vengono caricate in memoria GPU e il risultato  $C$  viene calcolato in parallelo dai thread CUDA.

#### 3.7.2 Script `matrixMul.slurm`

```
1 #SBATCH --partition=gpu
2 #SBATCH --gres=gpu:p100:1
3
4 module load cuda
5
6 WA=1024; HA=1024; WB=1024; HB=1024
7 ./matrixMul -wA=$WA -hA=$HA -wB=$WB -hB=$HB >> mm_GPU.$HOSTNAME
   .csv
```

#### 3.7.3 Risultati GPU

| Ambiente | Device                     | Tempo (ms) | GFLOP/s | Risultato       |
|----------|----------------------------|------------|---------|-----------------|
| Cluster  | NVIDIA Tesla P100 (cc 6.0) | 1.006      | 2135.34 | PASS            |
| PC       | –                          | –          | –       | Non disponibile |

Tabella 4: Prestazioni GPU CUDA, matrici  $1024 \times 1024$  singola precisione.

La GPU non è disponibile sul MacBook Air M3, che utilizza l'architettura Metal di Apple (senza supporto CUDA). Il test è stato eseguito unicamente sul cluster nella partizione `gpu` richiedendo una GPU NVIDIA Tesla P100 (architettura Pascal, compute capability 6.0).

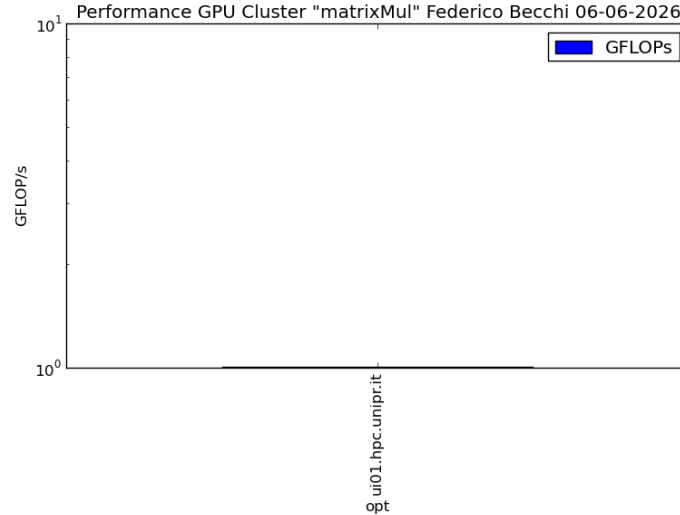


Figura 3: Prestazioni GPU CUDA (GFLOP/s), Cluster – NVIDIA Tesla P100.

### 3.8 Confronto CPU vs GPU

| Dispositivo | Configurazione migliore | GFLOP/s |
|-------------|-------------------------|---------|
| PC CPU      | OpenMP 8 thread         | 13.49   |
| Cluster CPU | icc + AVX + GSL         | 22.19   |
| Cluster GPU | NVIDIA Tesla P100 CUDA  | 2135.34 |

Tabella 5: Confronto prestazioni di picco ottenute con  $N = 1024$ .

La GPU offre un fattore di accelerazione di circa **96** $\times$  rispetto alla migliore configurazione CPU sul cluster, evidenziando il vantaggio dell’architettura SIMT per carichi di lavoro massicciamente paralleli.

## 4 Bandwidth e Latency di Rete

### 4.1 Descrizione

I benchmark `mpi_bandwidth.c` e `mpi_latency.c` misurano le prestazioni della rete di interconnessione usando scambi di messaggi MPI punto-a-punto:

- **Bandwidth:** invia messaggi di dimensione crescente (da 8 KB a 163 KB, step 8 KB) tra coppie di task MPI, misurando la banda in MB/s.
- **Latency:** scambia 100 messaggi da 1 byte con ping-pong tra 2 task, calcolando la latenza media in microsecondi.

Vengono testati due scenari:

- **Intra-nodo:** entrambi i task sullo stesso nodo (comunicazione via shared memory).
- **Inter-nodo:** un task per nodo (comunicazione via rete fisica OmniPath).



## 4.2 Script di esecuzione

### 4.2.1 PC – Docker band\_lat.sh

Sul MacBook Air mpirun non è disponibile nativamente. Il test è stato eseguito tramite due container Docker (`wn1`, `wn2`) sulla stessa macchina, collegati con una rete bridge. È stato necessario aggiungere gli hostname dei container nel `docker-compose.yml` per permettere la risoluzione dei nomi tra i due container.

```
1 mpicc mpi_bandwidth.c -o mpi_bandwidth
2 mpicc mpi_latency.c -o mpi_latency
3 mpirun -npernode 2 -np 2 -host wn1:4 mpi_bandwidth 2>
  band1-$HOSTNAME.csv
4 mpirun -npernode 1 -np 2 -host wn1:4,wn2:4 mpi_bandwidth 2>
  band2-$HOSTNAME.csv
5 mpirun -npernode 2 -np 2 -host wn1:4 mpi_latency 2>
  lat1-$HOSTNAME.csv
6 mpirun -npernode 1 -np 2 -host wn1:4,wn2:4 mpi_latency 2>
  lat2-$HOSTNAME.csv
```

I risultati ottenuti sui container Docker non sono rappresentativi di una rete fisica; il confronto significativo è quello tra intra-nodo e inter-nodo sul cluster HPC.

### 4.2.2 Cluster – band\_lat.slurm

```
1 #SBATCH --nodes=2
2 #SBATCH --ntasks-per-node=2
3 #SBATCH --constraint=omnipath
4
5 module load gnu8 openmpi3
6
7 mpicc mpi_bandwidth.c -o mpi_bandwidth
8 mpicc mpi_latency.c -o mpi_latency
9
10 mpirun -npernode 2 -np 2 mpi_bandwidth 2> band1-$HOSTNAME.csv
   # intra-nodo
11 mpirun -npernode 1 -np 2 mpi_bandwidth 2> band2-$HOSTNAME.csv
   # inter-nodo
12 mpirun -npernode 2 -np 2 mpi_latency 2> lat1-$HOSTNAME.csv
   # intra-nodo
13 mpirun -npernode 1 -np 2 mpi_latency 2> lat2-$HOSTNAME.csv
   # inter-nodo
```

Il job richiede 2 nodi sulla partizione `cpu_guest` con vincolo `omnipath` (interconnessione Intel OmniPath, 100 Gbps).

### 4.3 Risultati – Bandwidth

| Scenario               | Banda a 8 KB (MB/s) | Banda a 152 KB (MB/s) |
|------------------------|---------------------|-----------------------|
| Intra-nodo (wn20–wn20) | 1353                | 6015                  |
| Inter-nodo (wn20–wn21) | 2123                | 5159                  |

Tabella 6: Bandwidth MPI su cluster (OmniPath), nodo wn20.

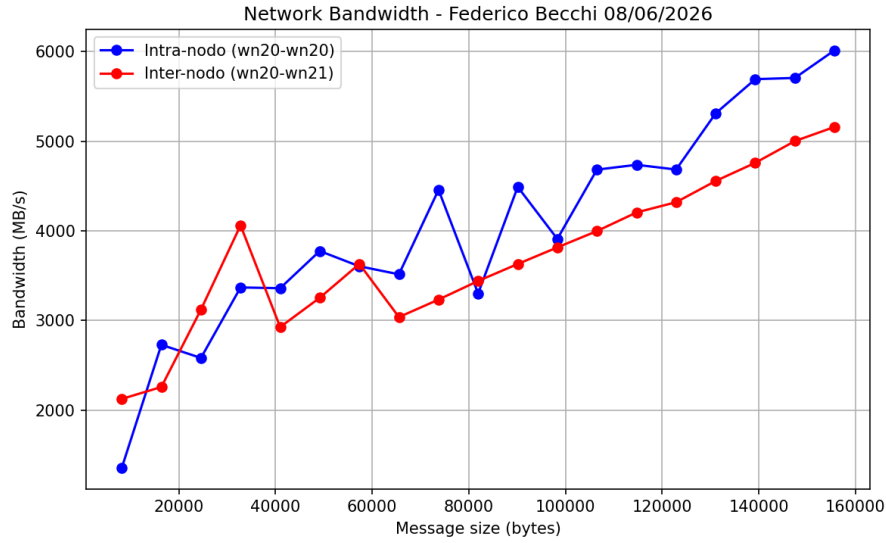


Figura 4: Bandwidth MPI in funzione della dimensione del messaggio – Cluster (OmniPath).

### 4.4 Risultati – Latency

| Scenario               | RTT medio ( $\mu s$ ) | Latenza one-way ( $\mu s$ ) |
|------------------------|-----------------------|-----------------------------|
| Intra-nodo (wn20–wn20) | 1.42                  | 0.71                        |
| Inter-nodo (wn20–wn21) | 2.33                  | 1.16                        |

Tabella 7: Latenza MPI (messaggi da 1 byte, 100 ping-pong), nodo wn20.

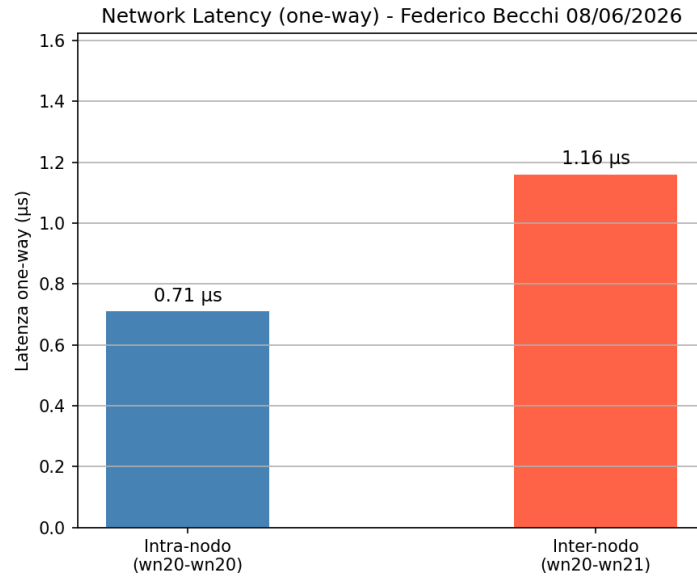


Figura 5: Latenza one-way MPI – Cluster (OmniPath).

## 4.5 Considerazioni

- La **banda intra-nodo** raggiunge 6015 MB/s ( $\approx 6$  GB/s) a messaggi grandi, sfruttando la memoria condivisa del nodo.
- La **banda inter-nodo** (rete OmniPath 100 Gbps) arriva a 5159 MB/s, leggermente inferiore a quella intra-nodo per i messaggi più grandi.
- A messaggi piccoli (8 KB) la situazione si inverte: l'inter-nodo misura 2123 MB/s contro i 1353 MB/s intra-nodo, probabilmente per il diverso protocollo usato da Open MPI per la shared memory (overhead di serializzazione per messaggi piccoli).
- La **latenza one-way** intra-nodo è  $0.71 \mu\text{s}$ , quella inter-nodo  $1.16 \mu\text{s}$ : entrambe nell'ordine del microsecondo, confermando le elevate prestazioni della rete OmniPath del cluster.

## 5 Disk Test

### 5.1 Descrizione

Il test misura le prestazioni di I/O del disco usando il comando `dd`, che scrive e poi legge un file temporaneo di dimensione configurabile:

```
1 dd if=/dev/zero of=$tempfile bs=1M count=$count && sync
2 dd if=$tempfile of=/dev/null bs=1M count=$count
```

Il comando `sync` forza il flush della cache del kernel sul disco prima di misurare il tempo, rendendo la misurazione di scrittura realistica. Sul cluster il test usa `count=2048` (2 GB); sul PC `count=512` (512 MB).

## 5.2 PC – disktest.sh

Lo script è stato adattato per macOS: l'opzione `conv=fdatasync` non è supportata dal `dd` di macOS, quindi è stata sostituita con `&& sync` che produce un effetto equivalente (flush del buffer del kernel). Il test scrive e legge un file da 512 MB (`count=512`).

| Operazione | Dimensione | Throughput           |
|------------|------------|----------------------|
| Scrittura  | 512 MB     | 962 MB/s             |
| Lettura    | 512 MB     | 3581 MB/s (3.5 GB/s) |

Tabella 8: Prestazioni I/O disco – PC (MacBook Air M3, SSD interno).

## 5.3 Cluster – disktest.slurm

Sul cluster il test viene eseguito su tre filesystem diversi:

- **\$HOME**: filesystem NFS condiviso tra tutti gli utenti.
- **\$SCRATCH**: filesystem ad alte prestazioni (Lustre/GPFS), ottimizzato per l'I/O parallelo nei job HPC.
- **\$ARCHIVE**: filesystem di archiviazione per grandi volumi di dati a bassa frequenza di accesso.

```

1 #SBATCH --partition=cpu_guest
2 #SBATCH --nodes=1
3
4 echo "#_HOME_perf"
5 cd $HOME && sh disktest.sh
6
7 echo "#_SCRATCH_perf"
8 cd $SCRATCH && sh disktest.sh
9
10 echo "#_ARCHIVE_perf"
11 cd $ARCHIVE && sh disktest.sh

```

## 5.4 Risultati

| Ambiente            | Filesystem  | Scrittura                             | Lettura  |
|---------------------|-------------|---------------------------------------|----------|
| PC (MacBook Air M3) | SSD interno | 962 MB/s                              | 3.5 GB/s |
| Cluster – \$HOME    | NFS         | 304 MB/s                              | 3.4 GB/s |
| Cluster – \$SCRATCH | Lustre      | 884 MB/s                              | 2.5 GB/s |
| Cluster – \$ARCHIVE | –           | non misurato (\$ARCHIVE non definito) |          |

Tabella 9: Prestazioni I/O disco: PC vs filesystem del cluster HPC.

## 5.5 Considerazioni

- La **scrittura** più rapida è quella del PC (962 MB/s): il SSD NVMe interno non è soggetto a contention e non attraversa la rete.
- **\$SCRATCH** (884 MB/s in scrittura) è il filesystem più performante sul cluster, confermando la sua vocazione all'I/O parallelo ad alto throughput.
- **\$HOME** (304 MB/s in scrittura) è più lento perché è un filesystem NFS condiviso: l'I/O passa dalla rete e è soggetto a contention tra tutti gli utenti del cluster.
- La **lettura** è molto rapida su tutti gli ambienti (2.5–3.5 GB/s) grazie alla cache del kernel (*page cache*): dopo la scrittura, i blocchi restano in memoria e la lettura non accede fisicamente al disco.
- La gerarchia di scrittura misurata è: SSD locale > SCRATCH  $\gg$  HOME, in linea con le aspettative architetturali.

## 6 Conclusioni

- Il test di **timing** ha confermato la distinzione tra wall-clock time e CPU time: la `sleep(1)` incrementa solo il REALTIME, non il CPUTIME. Il chip Apple M3 esegue il ciclo floating point più velocemente del nodo cluster Xeon (CPUTIME 0.37 s vs 0.97 s).
- Nella **moltiplicazione matriciale CPU**, l'uso di routine BLAS ottimizzate (GSL) porta a un aumento di prestazioni di oltre 14 $\times$  rispetto all'implementazione naive. Le istruzioni vettoriali AVX del cluster amplificano ulteriormente il vantaggio.
- Il parallelismo **OpenMP** scala quasi linearmente fino a 4 thread; oltre tale soglia emergono colli di bottiglia legati alla memoria condivisa.
- La **GPU NVIDIA Tesla P100** supera di circa 96 $\times$  la migliore CPU, grazie alle migliaia di core CUDA che eseguono le moltiplicazioni massivamente in parallelo (2135 GFLOP/s vs 22 GFLOP/s).
- I benchmark di **rete MPI** sul cluster mostrano una banda di picco di 6015 MB/s intra-nodo e 5159 MB/s inter-nodo (OmniPath), con latenze one-way rispettivamente di 0.71  $\mu$ s e 1.16  $\mu$ s.
- I test di **I/O disco** confermano la gerarchia attesa: il SSD locale del PC scrive a 962 MB/s; sul cluster, \$SCRATCH (884 MB/s) supera \$HOME (304 MB/s) perché quest'ultimo è un NFS condiviso soggetto a contention.